



C++ - Module 04

Subtype Polymorphism, Abstract Classes, and Interfaces

Summary: This document contains the exercises for Module 04 of the C++ modules.

Version: 12.0

Contents

I	Introduction	2
II	General rules	3
III	AI Instructions	6
IV	Exercise 00: Polymorphism	8
V	Exercise 01: I don't want to set the world on fire	10
VI	Exercise 02: Abstract class	12
VII	Exercise 03: Interface & recap	13
VIII	Submission and Peer Evaluation	17

Chapter I

Introduction

C++ is a general-purpose programming language created by Bjarne Stroustrup as an extension of the C programming language, or "C with Classes" (source: [Wikipedia](#)).

The goal of these modules is to introduce you to **Object-Oriented Programming**. This will serve as the starting point of your C++ journey. Many languages are recommended for learning OOP, but we've chosen C++ since it's derived from your old friend C. Because C++ is a complex language, and to keep things simple, your code will comply with the C++98 standard.

We are aware that modern C++ differs significantly in many aspects. Therefore, if you want to become a proficient C++ developer, it's up to you to continue your journey beyond the 42 Common Core!

Chapter II

General rules

Compiling

- Compile your code with `c++` and the flags `-Wall -Wextra -Werror`
- Your code should still compile if you add the flag `-std=c++98`

Formatting and naming conventions

- The exercise directories will be named this way: `ex00`, `ex01`, ..., `exn`
- Name your files, classes, functions, member functions and attributes as required in the guidelines.
- Write class names in **UpperCamelCase** format. Files containing class code will always be named according to the class name. For instance: `ClassName.hpp/ClassName.h`, `ClassName.cpp`, or `ClassName.hpp`. Then, if you have a header file containing the definition of a class "BrickWall" standing for a brick wall, its name will be `BrickWall.hpp`.
- Unless specified otherwise, every output message must end with a newline character and be displayed to the standard output.
- *Goodbye Norminette!* No coding style is enforced in the C++ modules. You can follow your favorite one. But keep in mind that code your peer evaluators can't understand is code they can't grade. Do your best to write clean and readable code.

Allowed/Forbidden

You are not coding in C anymore. Time to C++! Therefore:

- You are allowed to use almost everything from the standard library. Thus, instead of sticking to what you already know, it would be smart to use the C++-ish versions of the C functions you are used to as much as possible.
- However, you can't use any other external library. It means C++11 (and derived forms) and Boost libraries are forbidden. The following functions are forbidden too: `*printf()`, `*alloc()` and `free()`. If you use them, your grade will be 0 and that's it.

- Note that unless explicitly stated otherwise, the `using namespace <ns_name>` and `friend` keywords are forbidden. Otherwise, your grade will be -42.
- **You are allowed to use the STL only in Modules 08 and 09.** That means: no **Containers** (vector/list/map, and so forth) and no **Algorithms** (anything that requires including the `<algorithm>` header) until then. Otherwise, your grade will be -42.

A few design requirements

- Memory leakage occurs in C++ too. When you allocate memory (by using the `new` keyword), you must avoid **memory leaks**.
- From Module 02 to Module 09, your classes must be designed in the **Orthodox Canonical Form, except when explicitly stated otherwise**.
- Any function implementation put in a header file (except for function templates) means 0 to the exercise.
- You should be able to use each of your headers independently from others. Thus, they must include all the dependencies they need. However, you must avoid the problem of double inclusion by adding **include guards**. Otherwise, your grade will be 0.

Read me

- You can add some additional files if you need to (i.e., to split your code). As these assignments are not verified by a program, feel free to do so as long as you turn in the mandatory files.
- Sometimes, the guidelines of an exercise look short but the examples can show requirements that are not explicitly written in the instructions.
- Read each module completely before starting! Really, do it.
- By Odin, by Thor! Use your brain!!!



Regarding the Makefile for C++ projects, the same rules as in C apply (see the Norm chapter about the Makefile).



You will have to implement a lot of classes. This can seem tedious, unless you're able to script your favorite text editor.



You are given a certain amount of freedom to complete the exercises. However, follow the mandatory rules and don't be lazy. You would miss a lot of useful information! Do not hesitate to read about theoretical concepts.

Chapter III

AI Instructions

● Context

This project is designed to help you discover the fundamental building blocks of your ICT training.

To properly anchor key knowledge and skills, it's essential to adopt a thoughtful approach to using AI tools and support.

True foundational learning requires genuine intellectual effort — through challenge, repetition, and peer-learning exchanges.

For a more complete overview of our stance on AI — as a learning tool, as part of the ICT curriculum, and as an expectation in the job market — please refer to the dedicated FAQ on the intranet.

● Main message

- ✎ Build strong foundations without shortcuts.
- ✎ Really develop tech & power skills.
- ✎ Experience real peer-learning, start learning how to learn and solve new problems.
- ✎ The learning journey is more important than the result.
- ✎ Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

● Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.

- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

● Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

● Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter IV

Exercise 00: Polymorphism

	Exercise : 00
Polymorphism	
Turn-in directory: <i>ex00/</i>	
Files to turn in: <i>Makefile, main.cpp, *.cpp, *.{h, hpp}</i>	
Forbidden functions: <i>None</i>	

For every exercise, you have to provide the **most complete tests** you can. Constructors and destructors of each class must display specific messages. Don't use the same message for all classes.

Start by implementing a simple base class called **Animal**. It has one protected attribute:

- `std::string type;`

Implement a **Dog** class that inherits from **Animal**.

Implement a **Cat** class that inherits from **Animal**.

These two derived classes must set their **type** field depending on their name. Then, the Dog's type will be initialized to "Dog", and the Cat's type will be initialized to "Cat". The type of the Animal class can be left empty or set to the value of your choice.

Every animal must be able to use the member function:

`makeSound()`

It will print an appropriate sound (cats don't bark).

Running this code should print the specific sounds of the Dog and Cat classes, not the Animal's.

```
int main()
{
    const Animal* meta = new Animal();
    const Animal* j = new Dog();
    const Animal* i = new Cat();

    std::cout << j->getType() << " " << std::endl;
    std::cout << i->getType() << " " << std::endl;
    i->makeSound(); //will output the cat sound!
    j->makeSound();
    meta->makeSound();
    ...

    return 0;
}
```

To ensure you understood how it works, implement a **WrongCat** class that inherits from a **WrongAnimal** class. If you replace the Animal and the Cat by the wrong ones in the code above, the WrongCat should output the WrongAnimal sound.

Implement and turn in more tests than the ones given above.

Chapter V

Exercise 01: I don't want to set the world on fire

	Exercise : 01
I don't want to set the world on fire	
Turn-in directory: <i>ex01/</i>	
Files to turn in: Files from previous exercise + *.cpp, *.{h, hpp}	
Forbidden functions: None	

Constructors and destructors of each class must display specific messages.

Implement a **Brain** class. It contains an array of 100 `std::string` called `ideas`. This way, Dog and Cat will have a private `Brain*` attribute. Upon construction, Dog and Cat will create their Brain using `new Brain()`; Upon destruction, Dog and Cat will `delete` their Brain.

In your main function, create and fill an array of **Animal** objects. Half of it will be **Dog** objects and the other half will be **Cat** objects. At the end of your program execution, loop over this array and delete every Animal. You must delete directly dogs and cats as Animals. The appropriate destructors must be called in the expected order.

Don't forget to check for **memory leaks**.

A copy of a Dog or a Cat mustn't be shallow. Thus, you have to test that your copies are deep copies!

```
int main()
{
    const Animal* j = new Dog();
    const Animal* i = new Cat();

    delete j; //should not create a leak
    delete i;
    ...

    return 0;
}
```

Implement and turn in more tests than the ones given above.

Chapter VI

Exercise 02: Abstract class

	Exercise : 02
Abstract class	
Turn-in directory: <i>ex02/</i>	
Files to turn in: Files from previous exercise + *.cpp, *.{h, hpp}	
Forbidden functions: None	

Creating Animal objects doesn't make sense after all. It's true, they make no sound!

To avoid any possible mistakes, the default Animal class should not be instantiable. Fix the Animal class so that nobody can instantiate it. Everything should work as before.

If you want to, you can update the class name by adding an A prefix to Animal.

Chapter VII

Exercise 03: Interface & recap

	Exercise : 03
Interface & recap	
Turn-in directory: <i>ex03/</i>	
Files to turn in: <i>Makefile</i> , <i>main.cpp</i> , <i>*.cpp</i> , <i>*.{h, hpp}</i>	
Forbidden functions: <i>None</i>	

Interfaces don't exist in C++98 (not even in C++20). However, pure abstract classes are commonly called interfaces. Thus, in this last exercise, let's try to implement interfaces in order to make sure you understand this module.

Complete the definition of the following **AMateria** class and implement the necessary member functions.

```
class AMateria
{
    protected:
        [...]

    public:
        AMateria(std::string const & type);
        [...]

        std::string const & getType() const; //Returns the materia type

        virtual AMateria* clone() const = 0;
        virtual void use(ICharacter& target);
};
```

Implement the concrete classes for Materias: **Ice** and **Cure**. Use their names in lowercase ("ice" for Ice, "cure" for Cure) to set their types. Of course, their member function `clone()` will return a new instance of the same type (i.e., if you clone an Ice Materia, you will get a new Ice Materia).

The `use(ICharacter&)` member function will display:

- Ice: `"* shoots an ice bolt at <name> *"`
- Cure: `"* heals <name>'s wounds *"`

`<name>` is the name of the Character passed as a parameter. Don't print the angle brackets (`<` and `>`).



While assigning a Materia to another, copying the type doesn't make sense.

Write the concrete class **Character** which will implement the following interface:

```
class ICharacter
{
    public:
        virtual ~ICharacter() {}
        virtual std::string const & getName() const = 0;
        virtual void equip(AMateria* m) = 0;
        virtual void unequip(int idx) = 0;
        virtual void use(int idx, ICharacter& target) = 0;
};
```

The **Character** possesses an inventory of 4 slots, which means at most 4 Materias. The inventory is empty upon construction. They equip the Materias in the first empty slot they find, in the following order: from slot 0 to slot 3. If they try to add a Materia to a full inventory, or use/unequip a non-existent Materia, nothing should happen (but bugs are still forbidden). The `unequip()` member function must NOT delete the Materia!



Handle the Materias your character leaves on the floor as you like. Save the addresses before calling `unequip()`, or anything else, but don't forget that you have to avoid memory leaks.

The `use(int, ICharacter&)` member function will have to use the Materia at the `slot[idx]`, and pass the target parameter to the `AMateria::use` function.



Your character's inventory will be able to support any type of `AMateria`.

Your **Character** must have a constructor taking its name as a parameter. Any copy (using copy constructor or copy assignment operator) of a **Character** must be **deep**. During copy, the **Materias** of a **Character** must be deleted before the new ones are added to their inventory. Of course, the **Materias** must be deleted when a **Character** is destroyed.

Write the concrete class **MateriaSource** which will implement the following interface:

```
class IMateriaSource
{
    public:
        virtual ~IMateriaSource() {}
        virtual void learnMateria(AMateria*) = 0;
        virtual AMateria* createMateria(std::string const & type) = 0;
};
```

- **learnMateria(AMateria*)**

Copies the **Materia** passed as a parameter and stores it in memory so it can be cloned later. Like the **Character**, the **MateriaSource** can know at most 4 **Materias**. They are not necessarily unique.

- **createMateria(std::string const &)**

Returns a new **Materia**. The latter is a copy of the **Materia** previously learned by the **MateriaSource** whose type equals the one passed as parameter. Returns 0 if the type is unknown.

In a nutshell, your **MateriaSource** must be able to learn "templates" of **Materias** to create them when needed. Then, you will be able to generate a new **Materia** using just a string that identifies its type.

Running this code:

```
int main()
{
    IMateriaSource* src = new MateriaSource();
    src->learnMateria(new Ice());
    src->learnMateria(new Cure());

    ICharacter* me = new Character("me");

    AMateria* tmp;
    tmp = src->createMateria("ice");
    me->equip(tmp);
    tmp = src->createMateria("cure");
    me->equip(tmp);

    ICharacter* bob = new Character("bob");

    me->use(0, *bob);
    me->use(1, *bob);

    delete bob;
    delete me;
    delete src;

    return 0;
}
```

Should output:

```
$> clang++ -W -Wall -Werror *.cpp
$> ./a.out | cat -e
* shoots an ice bolt at bob *$
* heals bob's wounds *$
```

As usual, implement and turn in more tests than the ones given above.



You can pass this module without doing exercise 03.

Chapter VIII

Submission and Peer Evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your folders and files to ensure they are correct.

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behavior change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific timeframe is defined as part of the evaluation.

You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.



????????????? XXXXXXXXXX = \$3\$6b616b91536363971573e58914295d42